NORAIZ RANA

Task #15

Topic

React Components Architecture Best Practices

Date:

30 July 2025

SUBMITTED TO:

APPVERSE TECHNOLOGIES

📓 Learning Report: React Components Architecture & Best Practices

1. Introduction

React encourages a **component-based architecture**, where the UI is built using reusable, independent, and self-contained building blocks called **components**. A good component architecture improves **scalability**, **maintainability**, and **developer productivity**—especially in large applications.

In this learning module, I explored the principles of **structuring React components**, **file/folder organization**, and **best practices for styling and modularity**.

---

2. Component Types in React

React components can be divided into two main types:

a. Presentational (UI) Components

- Focus on **how things look**
- Usually receive data via **props**
- Stateless or only use local UI state

b. Container (Logic) Components

- Focus on **how things work**
- Handle data fetching, business logic
- May pass data to child components

This separation makes the code **clean, reusable, and easier to test**.

---

3. Recommended Component Folder Structure

To ensure clarity and modularity, each component should have its own folder with all related files:

```
/src
 └── components/
     └── Button/
         ├── Button.jsx
         ├── Button.module.css (or .scss)
         ├── Button.test.js
         └── index.js
```

This structure allows each component to manage:

- Its logic (`.jsx`)
- Its styling (CSS Module, Tailwind, or Styled Component)
- Its tests and documentation

---

4. Best Practices for Component Architecture

1. Small, Focused Components

- Each component should do one thing well.
- Break complex UIs into smaller subcomponents.

2. Reusable and Generic

- Avoid hard-coding values. Use props to customize components.
- Keep components flexible and reusable.

3. Prop Validation

- Use `PropTypes` or TypeScript to define expected props and data types.

4. Lift State Up

- When multiple components need shared data, **lift state up** to the nearest common ancestor.

5. Separation of Concerns

- Keep business logic (data fetching, state) separate from UI rendering.
- Use custom hooks for reusable logic (`useUserData()`, `useTheme()`).

6. Consistent Naming Conventions

- File and component names should be PascalCase (`UserProfile.jsx`, `ProductCard.jsx`)
- CSS class names should follow BEM or Tailwind conventions

---

5. Styling Best Practices with Architecture

- Use **CSS Modules** for scoped styles in small-to-medium projects
- Use **Tailwind CSS** for utility-first design with rapid development
- Use **Styled Components** or **Emotion** when dynamic styling is needed with logic
- Keep styles **inside the component folder**, not globally